École Polytechnique

PIANO — Musical Transformer

Antoine DELACOUR | Eloïse TRABUT

ABC Notation

Brief presentation. ABC notation is a text-based format used to represent symbolic musical scores. A tune is typically composed of a short header containing musical metadata such as time signature, default note length, and key signature, followed by a body encoding the sequence of notes with ASCII characters.
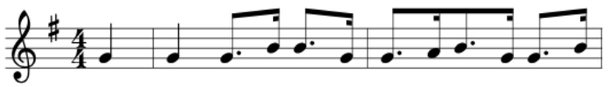
Advantages for machine learning. ABC notation is particularly convenient for generative models because music is represented directly as text. This makes symbolic music generation compatible with standard language-modeling methods such as autoregressive Transformers.

It also benefits from an existing ecosystem, with large datasets and conversion tools from other symbolic formats such as MIDI. In our project, we use the *Irish Massive ABC Notation Dataset*, which contains about 216k tunes and mostly monophonic melodies.

Example.

The following excerpt shows an ABC sequence and its corresponding score:

```
X:1
T:Generated Tune
M:4/4
L:1/8
K:G
G2 | G2 G>B B>G | G>AB>G G>B |
```



Tokenization in Music

Music generation models require musical sequences to be converted into discrete tokens before training.

The choice of tokenization strongly affects the length of sequences, training efficiency, and the ability of the model to capture musical structure.

Hard-coded tokenization.

Many symbolic music models rely on manually designed tokens aligned with musical concepts such as notes, durations, bar markers or ornaments. This approach incorporates musical knowledge directly into the representation.

For example, the *NotaGen* model uses a tokenization scheme specifically designed for ABC notation, where tokens correspond to musical events and structural symbols.

Learned token vocabulary.

In our approach, the tokenizer learns its vocabulary directly from the dataset using Byte Pair Encoding (BPE). The resulting alphabet contains both the basic characters used in ABC notation (letters, digits and punctuation such as `|`, `/`, `:`) and larger sub-sequences automatically discovered from the corpus.

Many learned tokens correspond to frequent musical fragments, such as short note patterns (e.g. `BAG`, `dcA`), bar symbols (`|:`), or common rhythmic structures. By merging frequently co-occurring characters, BPE produces tokens that represent recurring musical patterns.

This learned alphabet reduces sequence length compared to character-level tokenization and allows the model to capture recurring melodic motifs more efficiently.

However, because tokens are learned purely from statistical frequency rather than musical semantics, some tokens may correspond to arbitrary fragments that do not map cleanly to meaningful musical events.

This representation converts the original ABC sequences into token sequences that can be directly processed by an autoregressive Transformer model, which learns to predict the next token in the sequence.

Model

We formulate symbolic music generation as a sequence modeling task. Given a sequence of tokens derived from the ABC representation, the model learns to predict the next token in the sequence.

Our model.

Our architecture follows a GPT-style autoregressive Transformer. Each token is mapped to a learned embedding and processed through a stack of Transformer layers composed of self-attention and feed-forward blocks.

Self-attention enables the model to capture dependencies between notes across the sequence, allowing it to learn recurring patterns such as short melodic motifs or rhythmic structures.

The model is trained using a standard next-token prediction objective. Given a token sequence

$$t_1, t_2, \ldots, t_n$$

the model learns the conditional probability

$$P(t_{i+1} \mid t_1, \ldots, t_i)$$

New music is then generated by sampling tokens sequentially from the model.

Comparison with NotaGen.

The NotaGen model introduces a hierarchical generation strategy based on a double-decoder architecture. Instead of generating notes directly, the model first predicts higher-level musical semantics describing the structure of measures and then generates the corresponding note sequences.

This hierarchical design helps enforce stronger musical structure and improves the preservation of recurring musical themes.

In contrast, our model generates tokens directly with a single autoregressive Transformer. While this approach is simpler, it does not explicitly model higher-level musical structure, which can lead to weaker global organization and less consistent thematic development in generated pieces.

	Our Model	NotaGen
Architecture	GPT-style Transformer	Double decoder
Generation strategy	direct token generation	hierarchical generation
Structure modeling	implicit	explicit measure semantics
Theme consistency	limited	stronger

Data

We train our model on the *Irish Massive ABC Notation Dataset*, a large collection of traditional Irish folk tunes represented in ABC notation.

The dataset contains approximately 216 000 melodies. These pieces are mostly monophonic and follow relatively simple musical structures, which makes them particularly suitable for sequence modeling approaches.

Why this dataset.

ABC transcriptions are directly available and can therefore be used without complex symbolic conversion pipelines. In addition, the melodies typically consist of a single melodic line with limited harmonic complexity.

This simplicity makes the dataset well suited for our approach. Because our model generates music token by token without explicitly modeling higher-level musical structure, simpler monophonic melodies are easier to learn and reproduce.

It also allows us to obtain meaningful results using a relatively small Transformer model without requiring a very large architecture or extremely long training sequences.

Results

Model behaviour.

The model successfully learns several local musical patterns from the dataset. Generated sequences often follow valid ABC syntax and produce melodies that resemble the style of the training corpus.

However, the model also shows several limitations. Because generation is performed token by token without explicitly modeling higher-level musical structure, the global organization of the melody is often weak. In particular, the model may struggle to maintain consistent musical themes across multiple measures and sometimes produces repetitive patterns.

Memorization analysis.

To verify that the model does not simply copy melodies from the training dataset, we compared generated sequences with the original corpus.

Two tests were performed. First, we measured the maximum similarity between generated sequences and dataset entries. The highest observed similarity was approximately **0.46**. Second, we searched for the longest exact substring overlap between generated music and the training data.

The longest copied fragment was only **14 characters long**, corresponding to a short and common pattern in ABC notation.

These results suggest that the model does not memorize complete tunes from the dataset but instead recombines learned musical fragments to produce new melodies.

Example generation
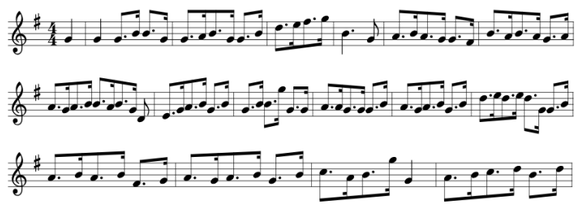
Generated ABC

```
X:1
T:Generated Tune
M:4/4
L:1/8
K:G
G2 | G2 G>B B>G | G>AB>G G>B | d>ef>g | B3 G |
A>BA>G G>F | B>AB>A G>A | A>GA>B B>AB>G D |
E>GA>B G>B | G>B B>g G>G | A>AG>G G>B |
A>GA>B G>B | d>ed>e d>G G>B | A>BA>B F>G |
A>GA>B G>B | c>AB>g G2 | A>Bc>d B>d |
```

Audio

Scan to listen

Score



References

NotaGen. Wang, Y., Wu, S., Hu, J., et al. *NotaGen: Advancing Musicality in Symbolic Music Generation with Large Language Model Training Paradigms.* arXiv:2502.18008, 2025.

NotaGen is a symbolic music generation model designed for ABC notation. It introduces a hierarchical generation strategy based on a double-decoder architecture. The model first predicts higher-level musical semantics describing the structure of measures, and then generates the corresponding sequence of notes. This approach improves global musical structure and helps preserve recurring melodic themes across longer passages.

Overview: Generating Music from Images

Page 1 showed how a language model trained on ABC notation can compose music from a text corpus. This page describes a more ambitious extension: **plAno generates music directly from an image**. The user uploads a photograph, and the system produces a melody whose mood reflects the visual content.

No paired dataset exists. Training such a system requires examples of the form (image, matching score). No such resource exists publicly. Our key insight is to use **CLIP** as a bridge. CLIP is a vision model trained to place images and their textual descriptions close together in the same 512-dimensional vector space. We therefore train on *text descriptions* of musical pieces rather than real images. At inference we simply swap in the *image embedding* — CLIP ensures that a photo of a stormy sea and the sentence "a dark Irish tune in D minor" land in the same neighbourhood of that space, so the model responds to the image exactly as it would to the description.

Why Mistral 7B? Mistral 7B is a powerful open-weight causal language model. Because ABC notation is plain text, it can be generated without any architectural change. Mistral 7B fits on a single Kaggle GPU after 4-bit compression, and LoRA fine-tuning specialises it to music while updating fewer than 1% of its parameters.

Building the Training Dataset

We reuse the same *Irish Massive ABC Notation Dataset* as page 1 (`sander-wood/irishman`, 10 000 tunes selected from 216 000). Exact duplicate tunes are removed before processing. Each remaining tune is converted into a training example through the following steps:

1 **Extract musical metadata.** Parse the ABC headers to retrieve the key (K:), meter (M:), and base note duration (L:). Tunes missing any of these fields are discarded.

2 **Assign an emotion label.** The musical mode determines the affect: minor, Dorian, Phrygian and Aeolian modes are labelled *sad and dark*; major mode is labelled *happy and upbeat*. This label will be embedded in the natural-language description.

3 **Build a rich text prompt.** A sentence is constructed that combines all extracted information: emotion, meter, base note, key, **and the structural control code**. The dataset provides a **control code** per tune: a compact structural description encoding how many times each section is played, which parts repeat, and the overall form of the piece. Example: "*A sad and dark Irish tune in 4/4, base note 1/8, key Emin, structure:* |:A|A|B|B:|" This prompt fully characterises the tune — not just its mood but also its rhythmic and formal organisation.

4 **Encode the prompt with CLIP.** The text prompt is encoded by CLIP ViT-B/32 in batches of 128 on GPU, producing an L2-normalised 512-dim vector. **This is the key step**: because CLIP aligns text and images in the same space, this vector acts as a *surrogate image* during training. The model learns to associate a 512-dim CLIP vector with the corresponding ABC score — without ever needing a real photo.

5 **Balance sad and happy classes.** The corpus is very unbalanced between "happy" and "sad" tunes. We downsample to $n_{\text{sad}} = n_{\text{happy}} = 2011$ so the model does not collapse to always producing the same type of melodies.

Structure of each training example. Each example stored in the JSONL dataset has two fields: `text` (the instruction string containing the string `<IMAGE_EMBED>`, and the ABC score as the expected output) and `image_features` (the 512 floats of the CLIP vector). These are two fundamentally different objects — one is a string to be tokenised, the other is a raw numerical vector to be injected into the model's embedding space. A standard training pipeline cannot handle both fields at once: it only knows how to process text. This is precisely **why a custom `MultimodalCollator` is needed**: it is the component that assembles these two heterogeneous fields into the single tensor representation that Mistral expects at each training step.

Finally, one of five instruction phrasings is randomly assigned per example (e.g. "Generate a traditional Irish tune based on this visual.") to prevent the model from memorising a single prompt form.

The MultimodalCollator: Assembling a Batch

The `SFTTrainer` calls the collator at every training step to transform a batch of raw examples into tensors ready for Mistral. A standard collator only handles text. Here, each example contains two heterogeneous objects — a text string and a 512-dim numerical vector — which a standard collator would either crash on or ignore. The `MultimodalCollator` replaces it and performs four operations in sequence on every batch:

1. Tokenise the text. The instruction + ABC score string is converted into a list of integer token IDs using Mistral's tokenizer. Because sequences in a batch have different lengths, shorter ones are padded with a dummy token to match the longest. The tokenizer simultaneously produces the **attention mask**: a binary vector (1 = real token, 0 = padding) that tells Mistral which positions to actually read and which to ignore. Without this mask, the model would treat padding as meaningful content and learn noise.

2. Convert token IDs into embedding vectors. Rather than letting Mistral do this step internally, the collator does it manually. Each integer token ID is looked up in Mistral's embedding table and replaced by its corresponding 4096-dimensional vector. The result is a tensor of shape (batch_size, sequence_length, 4096).

3. Inject the CLIP vector. The collator locates the position of `<IMAGE_EMBED>` in each sequence and **overwrites** the 4096-dim vector at that position with the pre-computed CLIP vector from `image_features`. Since CLIP produces 512-dim vectors, it is zero-padded to 4096: the first 512 values are the CLIP signal, the rest are zero. After this step, one slot in the embedding tensor no longer comes from Mistral's text vocabulary — it carries visual information.

4. The collator returns the embedding tensor. The trainer receives `inputs_embeds`, `attention_mask`, and `labels` (the original token IDs, used to compute the prediction loss).

The model then learns, step by step, to associate the visual signal sitting in those first 512 dimensions with the ABC musical content that follows.

Fine-tuning with LoRA

Instead of retraining Mistral's 7 billion parameters, we use **LoRA**: small trainable matrices are inserted into specific layers while the original weights stay frozen, updating fewer than 1% of parameters. The model is fine-tuned for 150 steps using `SFTTrainer`.

Choice	Value	Why
Quantisation	`NF4 4-bit`	Compresses 14 GB to ∼4 GB. `double_quant` saves a further 0.4 GB.
LoRA r / α	`16 / 32`	α=2r: standard stable scaling. r=16 balances capacity and memory.
LoRA dropout	`0.05`	Light regularisation to prevent overfitting on a small dataset.
Layers targeted	`attention + MLP`	Attention learns what to focus on; MLP layers encode musical style knowledge.
Optimiser	`paged_adamw_8bit`	Adam momentum offloaded to CPU memory, preventing GPU out-of-memory errors.
Learning rate	`1e-4, cosine`	Cosine schedule: starts at 1e-4 and decays smoothly, avoiding loss spikes at the end.
Warmup steps	`20`	Gradually ramps up LR to avoid instability at the start of training.
Batch / accum.	`2 / 4 = 8`	Effective batch of 8 gives a stable gradient signal within the GPU memory budget.
Max sequence	`512 tokens`	Covers ∼16 measures of ABC notation.

Generating Music from a New Image

At inference the user uploads one or more photos. Each image is encoded by CLIP into a 512-dim vector. When several images are provided, a slider lets the user set the influence weight of each; the vectors are merged as a weighted average:

$$\mathbf{e} = \frac{\sum_i w_i \, \text{CLIP}(\mathbf{I}_i)}{\sum_i w_i} \in \mathbb{R}^{512}$$

This vector is injected at `<IMAGE_EMBED>` and Mistral generates the ABC score token by token. The output passes through `nettoyer_abc` (header validation and normalisation), `exporter_midi` (music21 conversion with a fallback that retains only valid note characters), and finally FluidSynth renders a WAV file at 44 100 Hz.

Parameter	Value	Effect
`temperature`	`0.75`	Controls randomness. 0.75 is a good balance between creativity and musical coherence.
`repetition_penalty`	`1.2`	Discourages repeating the same note pattern, a common failure mode in LLMs.
`max_new_tokens`	`350`	Limits the score to ∼8–16 measures; longer outputs drift in quality.
`do_sample`	`True`	Stochastic sampling; essential because greedy decoding produces near-identical results every time.

Evaluating the Model

It is crucial to determine whether or not **fine-tuning make a difference.** To do so, in our code, the function `comparer_avec_et_sans_lora` runs the same image through the model twice with identical generation parameters: once with the LoRA adapters active (plAno), once with them disabled (base Mistral). The plAno model produces well-formed ABC scores with correct key, meter, and idiomatic Irish ornament syntax. The base model produces text that superficially resembles ABC notation but contains incoherent rhythms and harmonies — it has never been exposed to music during its pre-training.

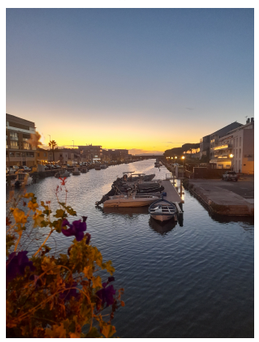
Generated Music Samples

Audio

Image n°1



Image n°2



References

Jiang et al. *Mistral 7B.* arXiv:2310.06825, 2023.

Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models.* 2021

Sander-Wood. *Irish Massive ABC Notation Dataset.* `sander-wood/irishman`, HuggingFace, 2023.

Qu et al. *MUPT: A Generative Symbolic Music Pre-trained Transformer.* arXiv:2404.06393, 2024.